

UNIVERSITÀ DEGLI STUDI DI BERGAMO

Scuola di Ingegneria

Corso di Laurea in Ingegneria Informatica

LABORATORIO DI SENSORI

SMART LAUNDRY COLOR CLASSIFIER

Sistema Embedded TinyML su Arduino Nano 33 BLE
per la Classificazione Cromatica e Notifica Wireless

Candidato:
Andrea Vaerini **1067398**

Docenti:
Prof. Gianluca Traversi
Prof. Paolo Lazzaroni

Anno Accademico 2026/2027

Indice

1 Introduzione e motivazione **3**
- 1.1 Automazione nello *Smart Laundry* . . . 3
- 1.2 Limiti dei sensori cromatici tradizionali . . . 3
- 1.3 Requisiti del progetto . . . 3
- 1.4 Sviluppo su singola scheda . . . 4

2 Architettura hardware e sensoristica integrata **4**
- 2.1 Arduino Nano 33 BLE Sense . . . 4
- 2.2 Il sensore Avago/Broadcom APDS-9960 . . . 4
  - 2.2.1 Canali RGBC . . . 5
  - 2.2.2 Rilevamento di prossimità . . . 5

3 Pipeline software e TinyML **6**
- 3.1 Normalizzazione cromatica . . . 6
- 3.2 Architettura del firmware . . . 6
  - 3.2.1 Modello TinyML con Edge Impulse . . . 6
  - 3.2.2 Stack Bluetooth Low Energy . . . 7
- 3.3 Macchina a stati a due stadi . . . 8

4 Analisi sperimentale e discussione delle criticità **9**
- 4.1 Metodologia di misura . . . 9
- 4.2 Stabilità delle letture al variare dell'illuminazione . . . 9
  - 4.2.1 Efficacia della normalizzazione . . . 9
  - 4.2.2 Limiti osservati: segnale basso e quantizzazione . . . 10
- 4.3 Ambiguità «nero contro vuoto» e Proximity Gate . . . 11
- 4.4 Valutazione del modello TinyML . . . 11
  - 4.4.1 Dataset e configurazione dell'addestramento . . . 11
  - 4.4.2 Risorse di calcolo e memoria . . . 12
  - 4.4.3 Validation set e matrice di confusione . . . 12
- 4.5 Collaudo end-to-end via BLE . . . 14

5 Conclusioni e sviluppi futuri **15**
- 5.1 Sintesi dei risultati . . . 15
- 5.2 Sviluppi futuri . . . 16
- 5.3 Prospettive di integrazione commerciale . . . 16

Bibliografia **16**

Elenco delle figure

2.1 Schema a blocchi funzionale dell'APDS-9960 e collegamento al SoC. 5
3.1 Macchina a stati del firmware: Proximity Gate seguito dalla classificazione. . . 8
4.1 Medie delle coordinate cromatiche nel piano $(r_{\text{norm}}, g_{\text{norm}})$. 10
4.2 Feature explorer di Edge Impulse: campioni del training set nello spazio $(r_{\text{norm}}, g_{\text{norm}}, b_{\text{norm}})$. 12
4.3 Matrice di confusione del modello float32 sul validation set (Edge Impulse). . . 13
4.4 Valori notificati via BLE durante il collaudo (nRF Connect). 14

Elenco delle tabelle

2.1 Specifiche principali della Arduino Nano 33 BLE Sense. 4
3.1 Librerie software utilizzate nel firmware. 6
3.2 Profilo GATT esposto dal dispositivo. 7
4.1 Statistiche delle letture per colore e condizione di illuminazione. 9
4.2 Occupazione di risorse del firmware e del modello. 12
4.3 Prestazioni on-device stimate da Edge Impulse (Cortex-M4F, 64 MHz, float32). 12
4.4 Metriche per classe sul validation set (modello float32). 13
4.5 Sintesi delle notifiche ricevute durante il collaudo BLE. 15

1. Introduzione e motivazione

1.1 Automazione nello *Smart Laundry*

La separazione dei capi per colore prima del lavaggio è un'operazione quotidiana tanto banale quanto soggetta a errore: un singolo indumento rosso o blu inserito in un carico di bianchi è sufficiente a compromettere l'intero lavaggio per effetto del rilascio di coloranti (*color bleeding*). Le lavatrici di fascia alta integrano già sensori di carico, di torbidità dell'acqua e di schiuma, ma il riconoscimento del colore dei capi resta affidato all'utente. Un sensore cromatico posto in prossimità dell'oblò, capace di avvisare l'utente direttamente sullo smartphone nel momento in cui inserisce un capo «incompatibile» con il programma selezionato, rappresenta un'estensione naturale del paradigma *Smart Home*.

1.2 Limiti dei sensori cromatici tradizionali

I classificatori di colore più semplici confrontano le tre componenti R, G, B fornite da un sensore con soglie fisse determinate empiricamente. Questo approccio presenta tre limiti principali:

- **Dipendenza dall'illuminazione**: l'intensità assoluta dei canali varia di un ordine di grandezza passando da luce diretta a ombra, per cui una soglia tarata in una condizione non è valida nelle altre;
- **Confini di decisione non lineari**: colori come rosa, viola e bianco occupano regioni dello spazio cromatico contigue e di forma irregolare, difficili da separare con semplici disuguaglianze;
- **Ambiguità del nero**: un tessuto nero opaco e l'assenza di qualunque oggetto davanti al sensore producono entrambi letture prossime allo zero su tutti i canali e risultano quindi indistinguibili sulla sola base dell'informazione cromatica.

Il progetto affronta il primo punto con una normalizzazione cromatica (Capitolo 3), il secondo sostituendo le soglie fisse con una piccola rete neurale addestrata (*TinyML*) e il terzo con un *Proximity Gate* basato sul sensore di prossimità a infrarossi integrato (Capitolo 4).

1.3 Requisiti del progetto

Il sistema sviluppato, denominato *Smart Laundry Color Classifier*, deve soddisfare i seguenti requisiti:

R1 – Classificazione multi-classe
Riconoscere otto classi cromatiche rappresentative dei capi domestici: `BLACK`, `WHITE`, `RED`, `BLUE`, `GREEN`, `PINK`, `PURPLE`, `YELLOW`.

R2 – Robustezza all'illuminazione
Fornire la stessa classificazione in luce diretta, in luce ambiente e in ombra.

R3 – Rilevamento della presenza
Distinguere un capo nero dall'assenza di capi, evitando falsi positivi quando il sensore è scoperto.

R4 – Inferenza a bordo
Eseguire l'intera elaborazione sul microcontrollore, senza dipendere da servizi cloud.

R5 – Notifica wireless
Inviare il risultato (classe e confidenza) a uno smartphone tramite Bluetooth Low Energy.

1.4 Sviluppo su singola scheda

Il progetto è stato svolto individualmente. Per contenere i tempi di sviluppo e massimizzare l'affidabilità meccanica del prototipo si è scelto di utilizzare esclusivamente la sensoristica già presente sulla scheda **Arduino Nano 33 BLE Sense**: il sensore di colore, prossimità e gesti APDS-9960 e il modulo radio BLE sono integrati sul PCB e collegati internamente al microcontrollore. Questa scelta elimina completamente cablaggi su breadboard, resistenze di pull-up esterne e problemi di contatti intermittenti, e consente di concentrare il lavoro sugli aspetti di elaborazione del segnale, di apprendimento automatico e di comunicazione. Il prototipo risultante è costituito da un unico modulo di 45 mm × 18 mm alimentato via USB.

2. Architettura hardware e sensoristica integrata

2.1 Arduino Nano 33 BLE Sense

La Arduino Nano 33 BLE Sense è una scheda di sviluppo in formato *Nano* costruita attorno al modulo u-blox NINA-B306, che ospita il SoC **Nordic Semiconductor nRF52840** [1]. Le caratteristiche rilevanti per il progetto sono riassunte in Tabella 2.1.

Tabella 2.1: Specifiche principali della Arduino Nano 33 BLE Sense.

Caratteristica	**Valore**
SoC	Nordic nRF52840
CPU	ARM Cortex-M4F, 64 MHz, FPU singola precisione
Memoria Flash	1 MB (983 040 B disponibili per lo sketch)
Memoria SRAM	256 kB
Radio	Bluetooth 5 / BLE, 2,4 GHz
Tensione di lavoro	3,3 V (alimentazione USB 5 V)
Bus sensori	I^2C interno (`Wire1`)
Sensori integrati	APDS-9960, IMU, microfono, temp./umidità, pressione

La presenza dell'unità in virgola mobile (FPU) del Cortex-M4F è particolarmente importante: permette di eseguire sia la normalizzazione cromatica sia l'inferenza della rete neurale direttamente in `float32`, senza ricorrere alla quantizzazione a 8 bit e quindi senza perdita di precisione numerica. Il firmware è basato sul core `Arduino Mbed OS Nano Boards` (versione 1.1.6), che fornisce lo stack BLE Cordio utilizzato dalla libreria `ArduinoBLE`.

2.2 Il sensore Avago/Broadcom APDS-9960

L'APDS-9960 [2] è un sensore digitale che integra in un unico package ottico quattro funzioni: misura di luce ambiente e colore (ALS/RGBC), rilevamento di prossimità, riconoscimento di gesti e un emettitore IR. Comunica tramite I^2C all'indirizzo fisso `0x39` (registro ID atteso: `0xAB`, verificato dal firmware in fase di diagnostica).

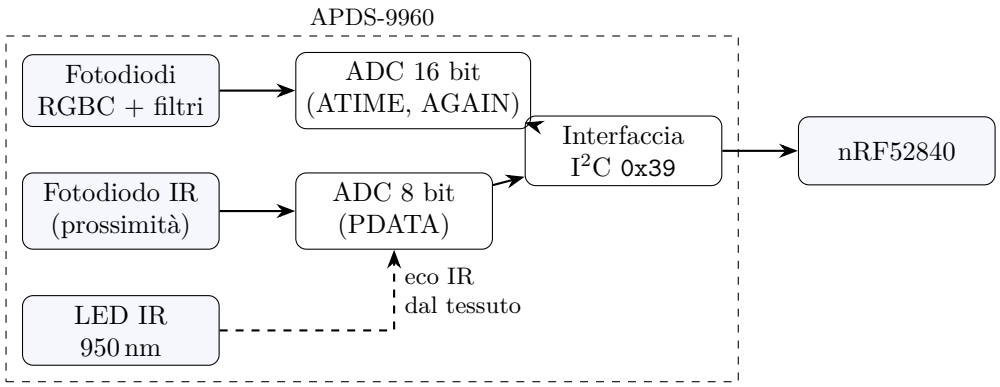


Figura 2.1: Schema a blocchi funzionale dell'APDS-9960 e collegamento al SoC.

2.2.1 Canali RGBC

Il blocco colore è costituito da una matrice di fotodiodi ricoperti da filtri interferenziali rosso, verde e blu, più un canale *Clear* senza filtro cromatico. Ciascun fotodiodo genera una fotocorrente proporzionale all'irradianza incidente pesata per la propria responsività spettrale $S_k(\lambda)$:

$$I_k = \int_{\lambda} S_k(\lambda)\, E(\lambda)\, \rho(\lambda)\, d\lambda, \qquad k \in \{R, G, B, C\} \tag{2.1}$$

dove $E(\lambda)$ è la distribuzione spettrale della sorgente luminosa e $\rho(\lambda)$ la riflettanza del tessuto. Le fotocorrenti sono integrate da quattro ADC a doppia rampa da 16 bit. Due registri determinano la sensibilità:

- **ATIME** (tempo di integrazione): ogni ciclo dura 2,78 ms e il valore massimo del conteggio vale $1025 \cdot (256 - \text{ATIME})$;
- **AGAIN** (guadagno analogico): selezionabile tra 1×, 4×, 16× e 64×.

La libreria `Arduino_APDS9960` imposta in fase di inizializzazione ATIME = 253 (3 cicli, circa 8,3 ms, fondo scala 3075 conteggi) e AGAIN = 16×. Come discusso nel Capitolo 4, questa configurazione, pensata per letture rapide in luce ambiente, determina conteggi molto bassi quando il tessuto è appoggiato sul sensore.

È importante sottolineare che l'APDS-9960 **non dispone di un illuminatore nel visibile**: le letture di colore dipendono interamente dalla luce ambiente riflessa dal tessuto, il che rende indispensabile la normalizzazione descritta nel Capitolo 3.

2.2.2 Rilevamento di prossimità

Il blocco di prossimità sfrutta un LED IR integrato (950 nm) che emette una serie di impulsi (configurati dalla libreria a 16 impulsi da 16 µs, registro `PPULSE = 0x8F`). La radiazione riflessa da un oggetto vicino viene raccolta da un fotodiodo IR dedicato; il segnale, depurato dalla componente ambiente, è convertito in un valore a 8 bit `PDATA` che **cresce all'avvicinarsi dell'oggetto**. La portata utile per superfici tessili è di pochi centimetri, ed è proprio questa caratteristica a rendere il sensore adatto a discriminare la presenza fisica di un capo indipendentemente dal suo colore: anche un tessuto nero riflette una frazione non nulla della radiazione nel vicino infrarosso.

3. Pipeline software e TinyML

3.1 Normalizzazione cromatica

Dall'Equazione (2.1) si osserva che, se la sorgente luminosa varia soltanto in intensità, ovvero $E(\lambda) \to k\,E(\lambda)$, tutti i canali vengono moltiplicati per lo stesso fattore k. Il rapporto tra un canale e la somma dei tre canali cromatici è quindi indipendente da k. Su questo principio si basano le **coordinate cromatiche normalizzate**, calcolate a partire dai conteggi grezzi R, G, B forniti dall'APDS-9960:

$$r_{\text{norm}} = \frac{R}{R+G+B}, \qquad g_{\text{norm}} = \frac{G}{R+G+B}, \qquad b_{\text{norm}} = \frac{B}{R+G+B}. \tag{3.1}$$

Per costruzione vale $r_{\text{norm}} + g_{\text{norm}} + b_{\text{norm}} = 1$: le tre feature giacciono su un piano (simplesso cromatico) e i gradi di libertà effettivi sono due. L'informazione sulla luminosità complessiva, che varia con l'illuminazione, viene scartata, mentre si conserva la *cromaticità*, legata alla riflettanza spettrale $\rho(\lambda)$ del tessuto. Il canale *Clear* non entra nelle feature ma viene registrato nelle misure come indicatore del livello di segnale.

Il caso degenere $R + G + B = 0$, che si presenta con tessuti neri o in completa oscurità, è gestito ponendo il denominatore pari a 1: il punto risultante $(0, 0, 0)$ si trova fuori dal simplesso ed è quindi facilmente riconoscibile dal classificatore, ma è *identico* per un capo nero e per l'assenza di capi. Questo aspetto è ripreso nella Sezione 4.3.

3.2 Architettura del firmware

Il firmware (`firmware/SmartLaundry/SmartLaundry.ino`) è scritto in C++ per l'ambiente Arduino e si appoggia a tre librerie principali, riassunte in Tabella 3.1.

Tabella 3.1: Librerie software utilizzate nel firmware.

Libreria	**Ruolo**
`Arduino_APDS9960`	Inizializzazione del sensore, lettura dei canali RGBC (`readColor`) e della prossimità (`readProximity`) via I^2C.
`Smart-Laundry-Color-Classifier_inferencing`	Libreria C++ generata da Edge Impulse: blocco DSP, rete neurale compilata con EON Compiler e funzione `run_classifier()`.
`ArduinoBLE`	Gestione del ruolo *peripheral* BLE: advertising, servizio GATT custom e notifiche.

3.2.1 Modello TinyML con Edge Impulse

L'addestramento è stato svolto sulla piattaforma Edge Impulse [3]. Per ogni classe sono stati acquisiti campioni delle tre coordinate normalizzate tramite porta seriale, nelle diverse condizioni di illuminazione descritte nel Capitolo 4. L'*impulse* è composto da:

Ingresso
Finestra di un singolo campione a tre assi (r_{norm}, g_{norm}, b_{norm}), frequenza nominale 10 Hz.

Blocco di elaborazione *Flatten*
Calcola la media di ciascun asse sulla finestra (unica statistica abilitata, scala 1,0), producendo un vettore di 3 feature.

Blocco di apprendimento *Classification*
Rete neurale densa con un livello nascosto da 16 neuroni ReLU e uno strato di uscita da 8 neuroni con attivazione *softmax*, per un totale di $(3 \cdot 16 + 16) + (16 \cdot 8 + 8) = 200$ parametri.

Il modello è stato esportato come libreria Arduino utilizzando l'**EON Compiler**, che, invece di includere un interprete TensorFlow Lite generico, genera direttamente codice C++ contenente il grafo, i pesi come array costanti in Flash e un'allocazione statica dei tensori. È stata scelta la variante **non ottimizzata (float32)**: data la dimensione ridottissima della rete (800 B di pesi), la quantizzazione `int8` non avrebbe portato benefici apprezzabili in memoria, mentre avrebbe introdotto errori di quantizzazione su feature già fortemente discretizzate (Sezione 4.2). L'arena dei tensori generata dal compilatore occupa soltanto 336 B di RAM.

```
int r_raw, g_raw, b_raw, c_raw;
APDS.readColor(r_raw, g_raw, b_raw, c_raw);

// Normalizzazione delle coordinate (r_norm, g_norm, b_norm)
float sum = (float)(r_raw + g_raw + b_raw);
if (sum == 0.0f) sum = 1.0f;

float r_norm = (float)r_raw / sum;
float g_norm = (float)g_raw / sum;
float b_norm = (float)b_raw / sum;

for (size_t ix = 0; ix < EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE; ix += 3) {
    features[ix + 0] = r_norm;
    features[ix + 1] = g_norm;
    features[ix + 2] = b_norm;
}

signal_t features_signal;
features_signal.total_length = EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE;
features_signal.get_data = &raw_feature_get_data;

ei_impulse_result_t result = { 0 };
EI_IMPULSE_ERROR res = run_classifier(&features_signal, &result, false);
```

Listato 3.1: Normalizzazione cromatica e inferenza (estratto da `loop()`).

Al termine dell'inferenza il firmware individua la classe con probabilità massima (*arg max* sull'uscita softmax) e costruisce il payload testuale nel formato `"CLASSE (NN%)"`, ad esempio `"BLUE (94%)"`.

3.2.2 Stack Bluetooth Low Energy

La scheda opera come *peripheral* GATT con nome `SmartLaundry-Nano33`. È stato definito un servizio custom contenente un'unica caratteristica di tipo stringa (massimo 64 byte) con proprietà `READ` e `NOTIFY`, così che lo smartphone riceva automaticamente ogni nuovo risultato senza effettuare *polling*.

Tabella 3.2: Profilo GATT esposto dal dispositivo.

Elemento	UUID	Proprietà
Servizio	`19B10000-E8F2-537E-4F6C-D104768A1214`	Primary
Caratteristica	`19B10001-E8F2-537E-4F6C-D104768A1214`	Read, Notify

```
BLEService laundryService("19B10000-E8F2-537E-4F6C-D104768A1214");
BLEStringCharacteristic colorResultChar(
```

```
    "19B10001-E8F2-537E-4F6C-D104768A1214", BLERead | BLENotify, 64);

BLE.setLocalName("SmartLaundry-Nano33");
BLE.setAdvertisedService(laundryService);
laundryService.addCharacteristic(colorResultChar);
BLE.addService(laundryService);
BLE.advertise();
```

Listato 3.2: Definizione del servizio GATT e avvio dell'advertising.

I payload possibili sono il nome di una delle otto classi seguito dalla confidenza percentuale, oppure il messaggio di stato `NESSUN_CAPO` generato dal Proximity Gate.

3.3 Macchina a stati a due stadi

Il comportamento del firmware è descritto dalla macchina a stati di Figura 3.1. Dopo l'inizializzazione, in cui il sensore viene avviato con un massimo di cinque tentativi e, in caso di fallimento, viene eseguita una scansione diagnostica del bus I^2C, il ciclo principale (periodo di circa 500 ms) è articolato in due stadi:

1. **Proximity Gate**: viene letto il valore di prossimità IR. Se nessun oggetto si trova entro circa 2–3 cm dal sensore, il sistema resta in *standby*, invia la notifica `NESSUN_CAPO` e non esegue alcuna classificazione;
2. **Classificazione TinyML**: solo se il capo è presente si acquisiscono i canali RGBC, si calcolano le coordinate normalizzate, si esegue l'inferenza e si notifica il risultato.

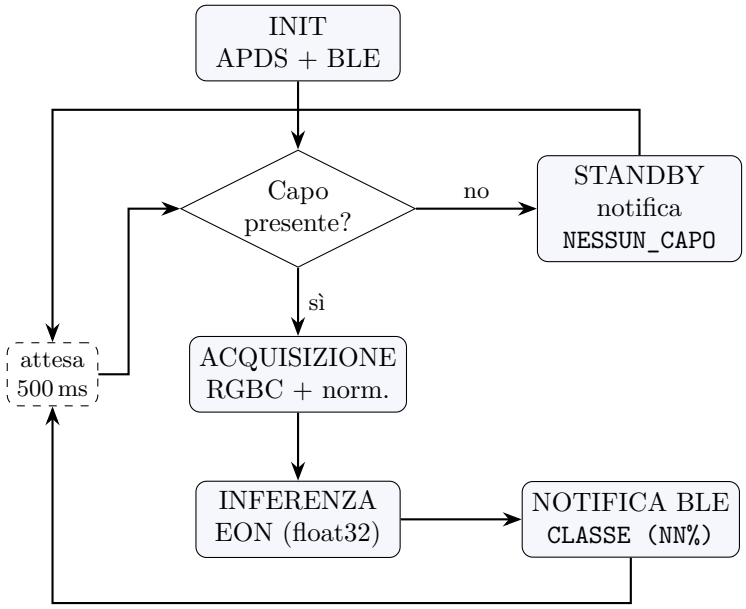


Figura 3.1: Macchina a stati del firmware: Proximity Gate seguito dalla classificazione.

La scelta di porre il gate *prima* dell'acquisizione cromatica ha un duplice vantaggio: elimina alla radice la classificazione di «capi fantasma» quando il sensore è scoperto e, in un'ottica di prodotto alimentato a batteria, consente di evitare l'esecuzione della rete neurale e la trasmissione di risultati privi di significato.

4. Analisi sperimentale e discussione delle criticità

4.1 Metodologia di misura

Per caratterizzare il comportamento del sensore è stato sviluppato uno sketch di acquisizione che stampa su porta seriale, in formato CSV, i conteggi grezzi R, G, B, C e le coordinate normalizzate dell'Equazione (3.1). Per ciascuno degli otto colori è stato utilizzato un campione di tessuto appoggiato sulla finestra ottica dell'APDS-9960, acquisendo da 9 a 14 letture consecutive in tre condizioni di illuminazione:

- **Luce diretta**: tessuto illuminato da una torcia/lampada puntata sul campione;
- **Luce ambiente**: normale illuminazione interna del laboratorio;
- **Ombra**: sensore e campione schermati dalla luce diretta.

I file grezzi (`report/data/<COLORE>_<condizione>.txt`) sono inclusi nel repository. La Tabella 4.1 riassume, per ogni combinazione, il numero di letture n, la media del canale *Clear* $\bar{C}$, le medie delle tre coordinate normalizzate e la deviazione standard campionaria σ_r di r_{norm}.

Tabella 4.1: Statistiche delle letture per colore e condizione di illuminazione.

Colore	**Condizione**	n	$\bar{C}$	$\bar{r}_{\text{norm}}$	$\bar{g}_{\text{norm}}$	$\bar{b}_{\text{norm}}$	σ_r
Nero	Luce diretta	10	0,7	0,000	0,000	0,000	0,000
	Luce ambiente	10	0,5	0,000	0,000	0,000	0,000
	Ombra	10	0,0	0,000	0,000	0,000	0,000
Bianco	Luce diretta	10	25,3	0,530	0,258	0,212	0,009
	Luce ambiente	11	3,3	0,539	0,261	0,200	0,070
	Ombra	12	4,2	0,531	0,240	0,229	0,046
Rosso	Luce diretta	9	3,8	0,596	0,280	0,123	0,067
	Luce ambiente	11	17,4	0,546	0,265	0,189	0,008
	Ombra	12	1,8	0,611	0,389	0,000	0,148
Blu	Luce diretta	10	2,7	0,366	0,316	0,316	0,070
	Luce ambiente	11	3,9	0,609	0,286	0,105	0,075
	Ombra	12	6,9	0,570	0,234	0,196	0,044
Verde	Luce diretta	12	27,8	0,431	0,309	0,260	0,045
	Luce ambiente	11	12,5	0,469	0,291	0,240	0,066
	Ombra	11	13,0	0,478	0,267	0,255	0,029
Rosa	Luce diretta	14	8,0	0,725	0,123	0,152	0,087
	Luce ambiente	11	2,3	0,964	0,018	0,018	0,121
	Ombra	11	2,6	0,927	0,036	0,036	0,162
Viola	Luce diretta	13	8,0	0,524	0,296	0,181	0,096
	Luce ambiente	11	3,9	0,463	0,278	0,260	0,057
	Ombra*	11	3,9	0,463	0,278	0,260	0,057
Giallo	Luce diretta	11	5,2	0,592	0,281	0,127	0,062
	Luce ambiente	10	4,0	0,600	0,233	0,167	0,061
	Ombra	10	4,4	0,583	0,225	0,192	0,063

* Serie identica a quella in luce ambiente: verosimile duplicazione in fase di salvataggio.

4.2 Stabilità delle letture al variare dell'illuminazione

4.2.1 Efficacia della normalizzazione

Il campione bianco costituisce il caso più chiaro: passando dalla luce diretta alla luce ambiente il canale *Clear* si riduce di quasi un fattore 8 (da 25,3 a 3,3 conteggi), mentre $\bar{r}_{\text{norm}}$ varia di meno

di 0,01 (0,530 contro 0,539) e le altre due coordinate di circa 0,02. Un comportamento analogo si osserva per il verde (variazione di $\bar{C}$ superiore a un fattore 2, scostamento massimo di $\bar{r}_{\text{norm}}$ pari a 0,047) e per il giallo (scostamento massimo 0,017). La normalizzazione rende quindi le feature largamente indipendenti dall'intensità della luce, come previsto dalla Sezione 3.1.

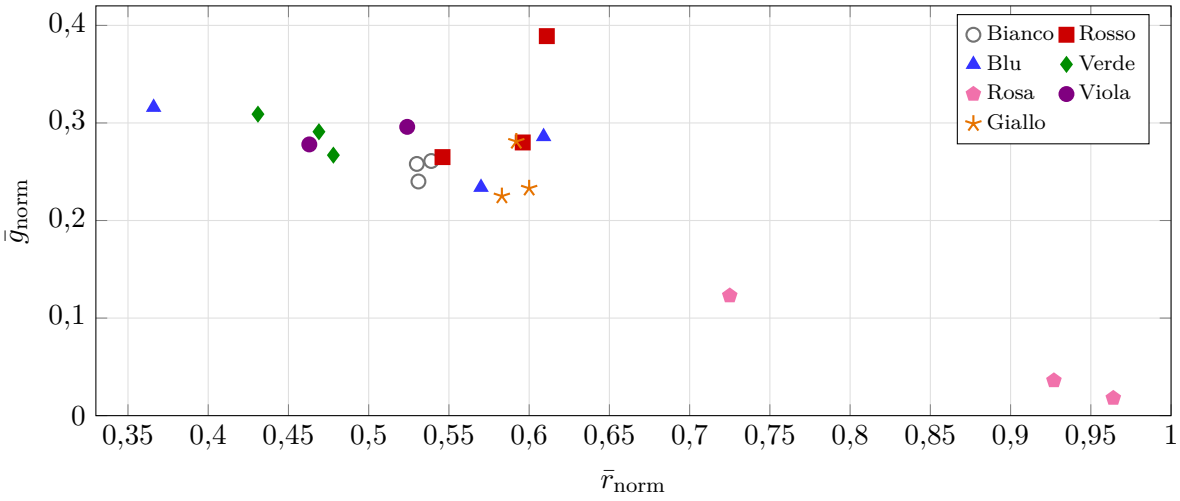


Figura 4.1: Medie delle coordinate cromatiche nel piano $(r_{\text{norm}}, g_{\text{norm}})$ per le tre condizioni di illuminazione. Il nero, sempre in $(0, 0)$, è fuori scala.

4.2.2 Limiti osservati: segnale basso e quantizzazione

L'analisi della Tabella 4.1 e della Figura 4.1 evidenzia però anche alcune criticità fisiche che è importante discutere.

Conteggi molto bassi
Con il tessuto appoggiato sulla finestra ottica, il campione stesso scherma gran parte della luce ambiente e, non essendoci un illuminatore nel visibile, il segnale che raggiunge i fotodiodi è minimo. Con la configurazione della libreria (8,3 ms, guadagno 16×, fondo scala 3075 conteggi), $\bar{C}$ non supera 28 conteggi, cioè meno dell'1 % della dinamica disponibile.

Quantizzazione delle feature
Con somme $R+G+B$ dell'ordine di 3–5 conteggi, le coordinate normalizzate possono assumere soltanto pochi valori discreti (0,250; 0,333; 0,500; 0,600; 0,667 . . .), come si vede chiaramente nei file grezzi. La risoluzione di r_{norm} è dunque dell'ordine di $1/(R+G+B) \approx 0{,}2$–$0{,}3$, e una singola unità di rumore sul canale blu sposta il punto di molti centesimi. Le serie con $\bar{C}$ elevato (bianco in luce diretta, rosso in luce ambiente) mostrano infatti $\sigma_r < 0{,}01$, mentre quelle a basso segnale arrivano a $\sigma_r \approx 0{,}15$.

Sbilanciamento verso il rosso
Anche il bianco presenta $r_{\text{norm}} \approx 0{,}53$ anziché il valore ideale 1/3. Ciò è dovuto alla combinazione della responsività spettrale dei fotodiodi, maggiore alle lunghezze d'onda lunghe, e dello spettro «caldo» delle sorgenti utilizzate. Lo sbilanciamento è sistematico e quindi viene appreso dal classificatore, ma riduce la distanza tra le classi.

Classi sovrapposte
Bianco, viola, verde, giallo e rosso occupano una regione compatta del piano cromatico ($r_{\text{norm}} \in [0{,}43;\ 0{,}61]$, $g_{\text{norm}} \in [0{,}22;\ 0{,}31]$). Il blu in particolare è ben distinto solo in luce diretta ($r_{\text{norm}} =$

0,366), mentre in luce ambiente e in ombra si avvicina al giallo. Al contrario rosa e nero sono separati nettamente da tutte le altre classi.

Queste osservazioni giustificano la scelta di un classificatore non lineare addestrato su dati raccolti in tutte le condizioni di illuminazione, anziché di semplici soglie, e anticipano i risultati della Sezione 4.5: confidenze elevate per le classi isolate e confidenze più basse per quelle che condividono la regione centrale del piano cromatico.

4.3 Ambiguità «nero contro vuoto» e Proximity Gate

Le misure sul campione nero mostrano $R = G = B = 0$ in tutte le letture e in tutte le condizioni di illuminazione: il tessuto nero opaco assorbe pressoché tutta la radiazione visibile e le coordinate normalizzate collassano nel punto $(0, 0, 0)$. La stessa lettura si ottiene però quando davanti al sensore non c'è alcun oggetto e la scena è poco illuminata (per esempio all'interno del cestello). Dal punto di vista del classificatore cromatico, quindi, «nero» e «nessun capo» sono la stessa classe: senza un'informazione aggiuntiva il sistema notificherebbe continuamente `BLACK` a sensore scoperto.

Soluzione adottata. L'informazione mancante è fornita dal canale di prossimità IR dello stesso APDS-9960. Il LED a 950 nm illumina attivamente la scena e la quantità di radiazione riflessa dipende principalmente dalla *distanza* della superficie, non dal suo colore nel visibile: un tessuto nero appoggiato al sensore produce un valore `PDATA` elevato, mentre l'aria non restituisce alcun eco. Il firmware considera presente un capo soltanto quando `PDATA` supera una soglia corrispondente a circa 2–3 cm; in caso contrario resta in standby e notifica `NESSUN_CAPO`.

La soglia è stata fissata sperimentalmente osservando il valore di prossimità stampato ogni 2 s sulla porta seriale nel messaggio di stato del firmware, mentre si avvicinavano progressivamente i campioni. Va notato che la libreria `Arduino_APDS9960` restituisce da `readProximity()` il valore complementare $255 - $ `PDATA`, crescente con la distanza; il confronto con la soglia deve quindi essere impostato tenendo conto di questa inversione di scala. Il gate a due stadi (Figura 3.1) separa così in modo netto i due casi: i falsi positivi `BLACK` a sensore scoperto vengono eliminati, come confermato dal collaudo BLE (Sezione 4.5).

4.4 Valutazione del modello TinyML

4.4.1 Dataset e configurazione dell'addestramento

Il training set è composto da 26 s di acquisizioni a 10 Hz, pari a 264 finestre di un campione ciascuna, distribuite sulle otto classi; Edge Impulse ne riserva una parte (42 campioni) come *validation set*. Le feature in uscita dal blocco Flatten non sono state ulteriormente normalizzate (opzione *Don't normalize data*), essendo già confinate in $[0, 1]$ dalla normalizzazione cromatica. La rete (ingresso a 3 feature, strato denso da 16 neuroni, uscita a 8 classi) è stata addestrata per 500 epoche con *learning rate* pari a 0,005 su CPU.

La Figura 4.2 mostra il *feature explorer* di Edge Impulse, cioè i campioni del training set nello spazio $(r_{\text{norm}}, g_{\text{norm}}, b_{\text{norm}})$. Si riconoscono il nero isolato nell'origine, il rosa spostato verso $r_{\text{norm}} \to 1$ e un ammasso centrale in cui bianco, viola, giallo, rosso e blu si sovrappongono, in accordo con la Figura 4.1.

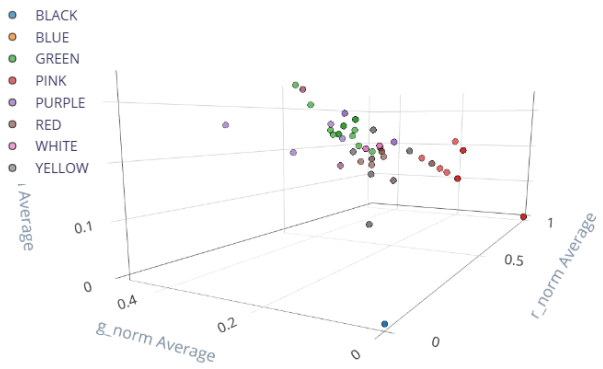


Figura 4.2: Feature explorer di Edge Impulse: campioni del training set nello spazio $(r_{\text{norm}}, g_{\text{norm}}, b_{\text{norm}})$.

4.4.2 Risorse di calcolo e memoria

La Tabella 4.2 riporta l'occupazione di memoria dell'intero firmware, ricavata dal report di compilazione, e le dimensioni del modello ricavate dal codice generato dall'EON Compiler; la Tabella 4.3 confronta le stime *on-device* fornite da Edge Impulse per la Nano 33 BLE Sense con i due motori di inferenza disponibili.

Tabella 4.2: Occupazione di risorse del firmware e del modello.

Voce	**Valore**	**Utilizzo**
Flash del firmware completo	324 584 B	33 % di 983 040 B
RAM statica (variabili globali)	69 624 B	26 % di 262 144 B
Pesi della rete (float32)	800 B	200 parametri
Arena dei tensori (EON)	336 B	–
Operazioni per inferenza	176 MAC	2 `FULLY_CONNECTED` + `SOFTMAX`

Tabella 4.3: Prestazioni on-device stimate da Edge Impulse (Cortex-M4F, 64 MHz, float32).

Motore	**Tempo di inferenza**	**Picco di RAM**	**Flash**
TensorFlow Lite Micro	1 ms	2,7 kB	33,8 kB
EON Compiler	1 ms	1,2 kB	11,3 kB
Riduzione con EON	–	−56 %	−67 %

L'EON Compiler, eliminando l'interprete generico e allocando staticamente i tensori, riduce di oltre la metà sia la RAM sia la Flash richieste dal modello, a parità di tempo di inferenza (1 ms, la risoluzione minima riportata dallo strumento). In ogni caso la quasi totalità della memoria del firmware è occupata dallo stack BLE di Mbed OS e dalle librerie di sistema: il contributo del modello è trascurabile e la latenza complessiva del sistema è dominata dal periodo del ciclo principale (500 ms) e dall'intervallo di connessione BLE, non dal calcolo.

4.4.3 Validation set e matrice di confusione

Sul validation set il modello float32 raggiunge un'accuratezza del **61,9 %** con *loss* pari a 1,11, area sotto la curva ROC di 0,86 e precisione, richiamo e F1 medi pesati pari rispettivamente a 0,57, 0,62 e 0,59. La variante quantizzata `int8`, valutata sugli stessi dati, scende al 57,1 %: il

confronto conferma sperimentalmente la scelta della versione non ottimizzata (Capitolo 3). La matrice di confusione è riportata in Figura 4.3 e le metriche per classe in Tabella 4.4.

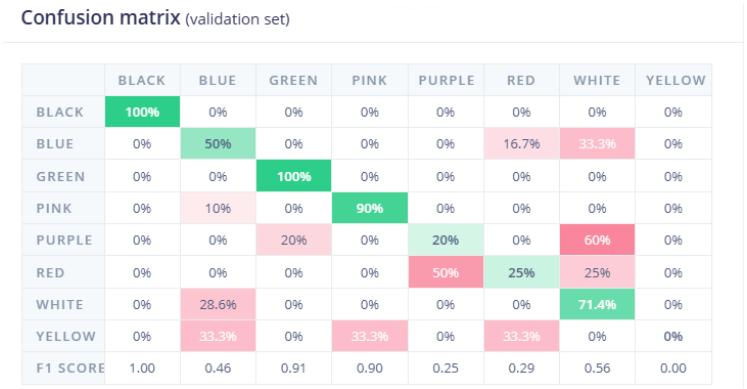


Figura 4.3: Matrice di confusione del modello float32 sul validation set (Edge Impulse).

Tabella 4.4: Metriche per classe sul validation set (modello float32).

Classe	**Campioni**	**Precisione**	**Richiamo**	**F1**
BLACK	2	1,00	1,00	1,00
BLUE	6	0,43	0,50	0,46
GREEN	5	0,83	1,00	0,91
PINK	10	0,90	0,90	0,90
PURPLE	5	0,33	0,20	0,25
RED	4	0,33	0,25	0,29
WHITE	7	0,45	0,71	0,56
YELLOW	3	0,00	0,00	0,00
Media pesata	42	0,57	0,62	0,59

Nero, verde e rosa sono riconosciuti in modo affidabile (F1 $\geq$ 0,90). Le confusioni principali sono viola→bianco (60 %), rosso→viola (50 %), blu→bianco (33 %) e bianco→blu (29 %), mentre i tre campioni gialli sono ripartiti fra blu, rosa e rosso. Si tratta esattamente delle classi che nella Tabella 4.1 condividono valori medi di $r_{\text{norm}} \approx 0{,}5$–$0{,}6$ e $g_{\text{norm}} \approx 0{,}25$. L'accuratezza di validazione è sensibilmente inferiore alla resa osservata nelle prove reali (Sezione 4.5); la discrepanza ha due cause principali:

- **Dimensione del validation set**: 42 campioni suddivisi su otto classi significano da 2 a 10 esempi per classe, per cui un singolo errore sposta il richiamo di classe di 10–50 punti percentuali e la stima è affetta da una varianza molto elevata. Il caso limite è il giallo: 0 % su soli 3 campioni di validazione, ma classificato correttamente nel collaudo;
- **Campioni quantizzati e ripetuti**: a causa della discretizzazione discussa nella Sezione 4.2, colori diversi in condizioni a basso segnale producono *esattamente* lo stesso vettore di feature (per esempio (0,5; 0,25; 0,25) compare per bianco, blu, giallo e viola). Questi campioni sono intrinsecamente non separabili e penalizzano la metrica, mentre nell'uso reale il capo viene presentato con un'illuminazione migliore e la lettura viene ripetuta ogni 500 ms.

4.5 Collaudo end-to-end via BLE

Il collaudo finale è stato svolto con la scheda alimentata via USB e uno smartphone Android con l'app **nRF Connect for Mobile** di Nordic Semiconductor. Dopo la scansione, il dispositivo `SmartLaundry-Nano33` (indirizzo `F5:07:A5:01:56:92`) è stato connesso senza *bonding*, sono state abilitate le notifiche sulla caratteristica `19b10001-...` scrivendo il descrittore CCCD (`0x2902`) e sono stati presentati in sequenza i campioni di tessuto. La Figura 4.4 riporta il dettaglio della caratteristica letta dall'app per ciascuna prova; i risultati sono riassunti in Tabella 4.5.



(a) `BLUE (94%)` (b) `BLUE (99%)` (c) `PINK (99%)`

(d) `PINK (99%)` (e) `BLACK (99%)` (f) `YELLOW (87%)`

(g) `RED (72%)` (h) `PURPLE (78%)` (i) `PURPLE (42%)`

(j) `GREEN (51%)` (k) `GREEN (46%)` (l) `WHITE (34%)`

(m) `NESSUN_CAPO` (n) `NESSUN_CAPO`

Figura 4.4: Valori notificati sulla caratteristica `19B10001` durante il collaudo (campo *Value*: byte esadecimali e stringa ASCII).

In tutte le prove documentate la classe con probabilità massima coincide con il colore del campione presentato e, a sensore scoperto, il sistema ha notificato correttamente `NESSUN_CAPO` anziché `BLACK`. I livelli di confidenza confermano quanto previsto dall'analisi del piano cromatico:

- le classi più separate (`BLUE` in buona illuminazione, `PINK`, `BLACK`) sono riconosciute con confidenza superiore al 90 %, e il giallo con l'87 %;
- le classi della regione centrale (`RED`, `PURPLE`, `GREEN`, `WHITE`) sono anch'esse classificate correttamente, ma con un margine ridotto (34–78 %), segno che la probabilità è ripartita fra più classi vicine. Il bianco al 34 % supera di poco la soglia di 0,3 definita nei metadati del modello (`EI_CLASSIFIER_THRESHOLD`), che il firmware attualmente non applica, limitandosi all'*arg max*.

Tabella 4.5: Sintesi delle notifiche ricevute durante il collaudo BLE.

Campione presentato	**Payload ricevuto**	**Confidenza**
Tessuto blu	`BLUE`	94 %, 99 %
Tessuto rosa	`PINK`	99 %, 99 %
Tessuto nero	`BLACK`	99 %
Tessuto giallo	`YELLOW`	87 %
Tessuto rosso	`RED`	72 %
Tessuto viola	`PURPLE`	78 %, 42 %
Tessuto verde	`GREEN`	51 %, 46 %
Tessuto bianco	`WHITE`	34 %
Sensore scoperto	`NESSUN_CAPO`	–

Il canale BLE si è dimostrato affidabile: i payload, lunghi al massimo 14 caratteri, rientrano in una singola notifica ATT anche con la MTU minima di 23 byte, e la connessione è rimasta stabile per tutta la durata della sessione.

5. Conclusioni e sviluppi futuri

5.1 Sintesi dei risultati

Il progetto ha portato alla realizzazione di un classificatore cromatico di tessuti completamente autonomo, basato su un'unica scheda Arduino Nano 33 BLE Sense e privo di hardware esterno. Rispetto ai requisiti del Capitolo 1:

R1, R4 – Classificazione a bordo
Una rete neurale densa da 200 parametri, generata con Edge Impulse ed EON Compiler in float32, distingue otto classi cromatiche occupando 800 B di pesi e 336 B di arena, con un tempo di inferenza trascurabile rispetto al periodo di aggiornamento.

R2 – Robustezza all'illuminazione
Le coordinate cromatiche normalizzate hanno reso le feature sostanzialmente indipendenti dall'intensità della luce: per il bianco, a fronte di una variazione di quasi 8 volte del segnale, r_{norm} varia di meno di 0,01.

R3 – Rilevamento della presenza
Il Proximity Gate basato sul canale IR dell'APDS-9960 ha risolto l'ambiguità fisica tra tessuto nero e assenza di capi, che l'informazione cromatica da sola non può distinguere.

R5 – Notifica wireless
Il servizio GATT custom con caratteristica `NOTIFY` ha consegnato in modo affidabile allo smartphone sia le classificazioni (es. `BLUE (99%)`, `PINK (99%)`, `BLACK (99%)`) sia lo stato `NESSUN_CAPO`.

L'analisi sperimentale ha inoltre messo in luce il limite principale del prototipo: in assenza di un illuminatore nel visibile e con la configurazione predefinita del sensore, il segnale con il tessuto a contatto è di pochi conteggi, e la conseguente quantizzazione delle feature spiega sia le confidenze ridotte sulle classi centrali del piano cromatico sia la modesta accuratezza misurata sul validation set (61,9 %).

5.2 Sviluppi futuri

- **Configurazione del sensore**: aumentare il tempo di integrazione (ATIME) fino a da 100 ms a 200 ms e il guadagno a 64× porterebbe i conteggi da qualche unità a qualche centinaio, riducendo di due ordini di grandezza l'errore di quantizzazione delle coordinate normalizzate.
- **Illuminazione attiva**: un LED bianco ad alta resa cromatica (CRI > 90) affiancato al sensore renderebbe la misura indipendente dalla luce ambiente, che all'interno di una lavatrice è di fatto assente.
- **Bilanciamento del bianco**: una calibrazione iniziale su un riferimento bianco permetterebbe di compensare lo sbilanciamento verso il rosso osservato (Sezione 4.2).
- **Dataset e robustezza**: ampliare il dataset con più tessuti per classe (trame, materiali, colori misti) e un validation set bilanciato, e applicare in firmware la soglia di confidenza per notificare `INCERTO` quando la probabilità massima è bassa.
- **Ottimizzazione energetica**: sfruttare l'interrupt di prossimità dell'APDS-9960 per mantenere il SoC in *sleep* fino all'avvicinamento di un capo, in vista di un'alimentazione a batteria.

5.3 Prospettive di integrazione commerciale

In un prodotto reale il modulo potrebbe essere integrato nella guarnizione o nel vetro dell'oblò, dietro una finestra ottica trasparente nel visibile e nel vicino infrarosso. All'inserimento di ciascun capo il Proximity Gate attiverebbe la lettura, e l'elettronica della lavatrice, o direttamente l'app dello smartphone, potrebbe confrontare il colore con il programma selezionato e segnalare in tempo reale un capo incompatibile (per esempio un capo rosso in un lavaggio di bianchi a 60 °C). Il costo della soluzione è contenuto, dato che sensore e SoC BLE sono componenti di largo consumo, e l'inferenza a bordo evita qualunque dipendenza da servizi cloud, garantendo il funzionamento anche in assenza di connettività Internet.

Bibliografia

[1] Nordic Semiconductor, *nRF52840 Product Specification v1.7*, 2021.

[2] Broadcom (Avago Technologies), *APDS-9960 Digital Proximity, Ambient Light, RGB and Gesture Sensor – Data Sheet*, AV02-4191EN, 2015.

[3] S. Hymel *et al.*, *Edge Impulse: An MLOps Platform for Tiny Machine Learning*, Proceedings of Machine Learning and Systems (MLSys), 2023.

[4] Arduino, *Arduino_APDS9960 Library*, `https://github.com/arduino-libraries/Arduino_APDS9960`.